

Data Mining Documentation

Step 1: Understand the data set features

Diagnosis: The target label —

- M or 1 = Malignant (cancerous)
 - B or 0 = Benign (non-cancerous)
-

Dataset Overview: This breast cancer diagnostic dataset classifies tumours as malignant (M/1) or benign (B/0) using ten core cellular characteristics, each measured in three ways (mean, standard error, and worst/maximum values). Features include: (1) radius (cell size), (2) texture (nuclei graininess), (3) perimeter (border length), (4) area, (5) smoothness (edge regularity), (6) compactness (shape density), (7) concavity (inward curve depth), (8) concave points (number of indentations), (9) symmetry, and (10) fractal dimension (edge complexity). Malignant tumours typically exhibit larger, irregular cells with spiky perimeters, deep concavities, and asymmetric/jagged edges, while benign tumours tend to have circular symmetry with smoother boundaries. The three versions of each measurement help show typical values (mean), how consistent they are (standard error), and the most concerning findings (worst values).

Step 2: Data Processing Step

- There are no duplicates, and no null cells/columns/rows.
- We split the data into training and testing set using 80% for train and 20% for test. Each of both have 2 sets (x is for the input / independent variables and y is for the dependant variable / the one we are trying to predict)
- `random_state=42`: Sets a seed for the random number generator used to shuffle and split the data. Machine learning datasets are often shuffled before splitting to ensure the training and test sets are representative of the overall data distribution. Setting `random_state=42` ensures reproducibility: every time you run the code with the same data and `random_state`, you get the same split.

Step 3: Data Classification step

We normalized / standardized the data so that we can have them all in the same range. We got the z – score for all the values and entries so that we can fit the data into our range.

We used 7 classification models.

- XG Boost - extra
 - Logistic Regression
 - Random Forest Classifier – extra
 - Support Vector Machine (SVM) - extra
 - k-Nearest Neighbors (KNN)
 - Decision Tree
 - Gaussian Naive Bayes
-
- ◆ **Logistic Regression:** Simple and interpretable baseline for binary classification.
 - ◆ **KNN:** Captures local patterns; useful for non-linear data with clear distance relationships.
 - ◆ **Decision Tree:** Easy to interpret; models hierarchical decision rules but prone to overfitting.
 - ◆ **Random Forest:** Ensemble of decision trees; robust with built-in feature importance and reduced overfitting.
 - ◆ **SVM (Support Vector Machine):** Effective for high-dimensional data; finds optimal decision boundaries via kernels.
 - ◆ **Gaussian Naive Bayes:** Fast probabilistic model; assumes feature independence but works well for many real-world tasks.
 - ◆ **XGBoost:** High-performance gradient boosting; optimizes speed and accuracy with regularization for complex data.

Results of the 7 classification models:

■ Feature Importance Plots - for Tree-Based Models

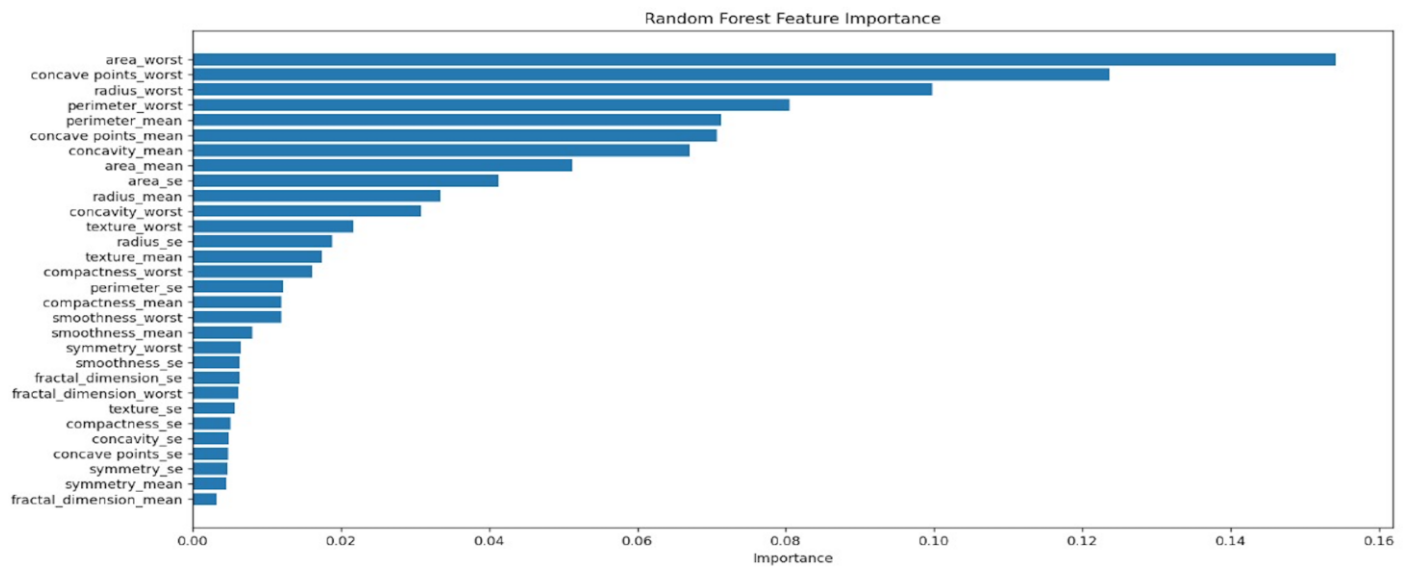
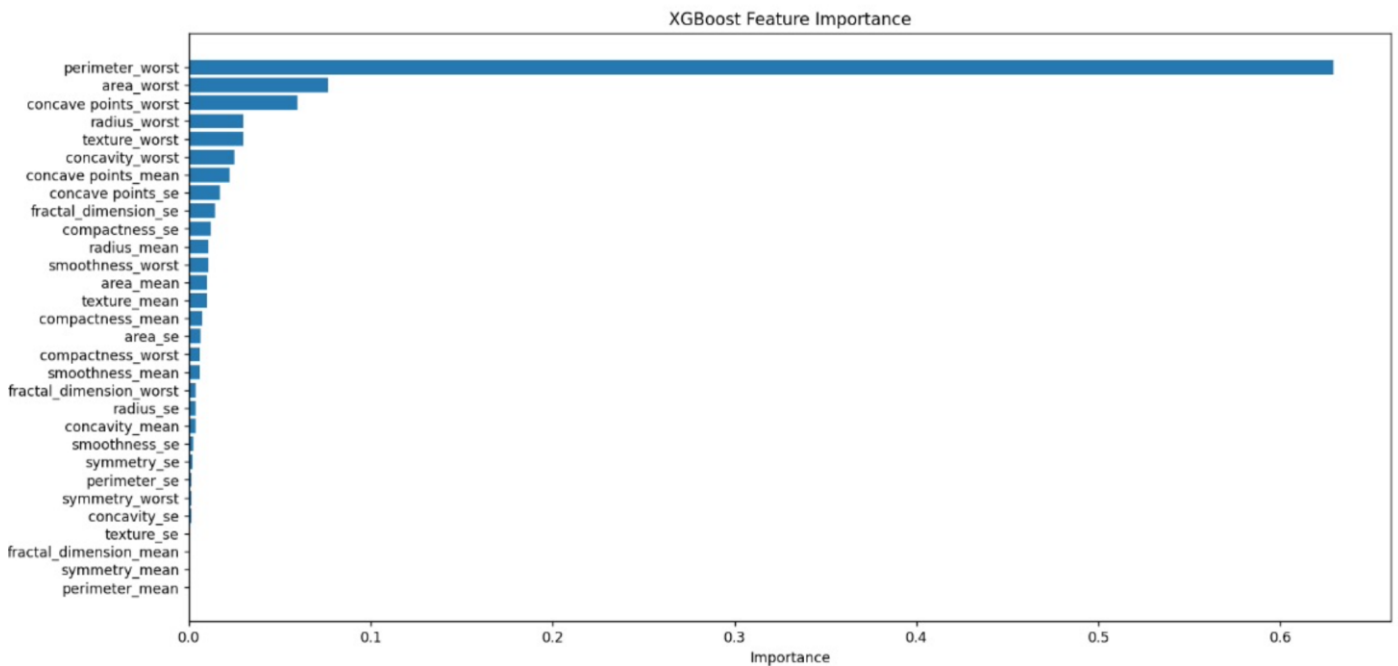
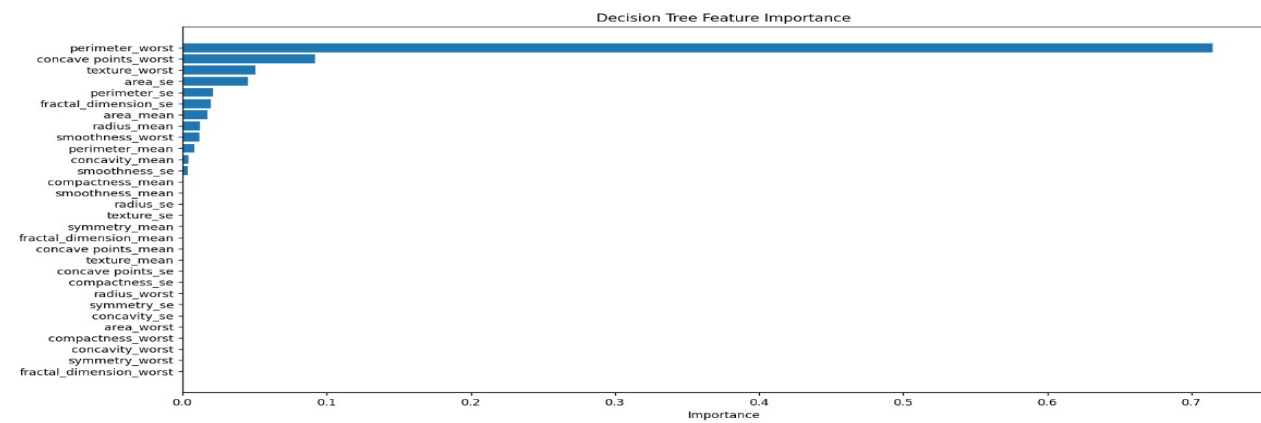
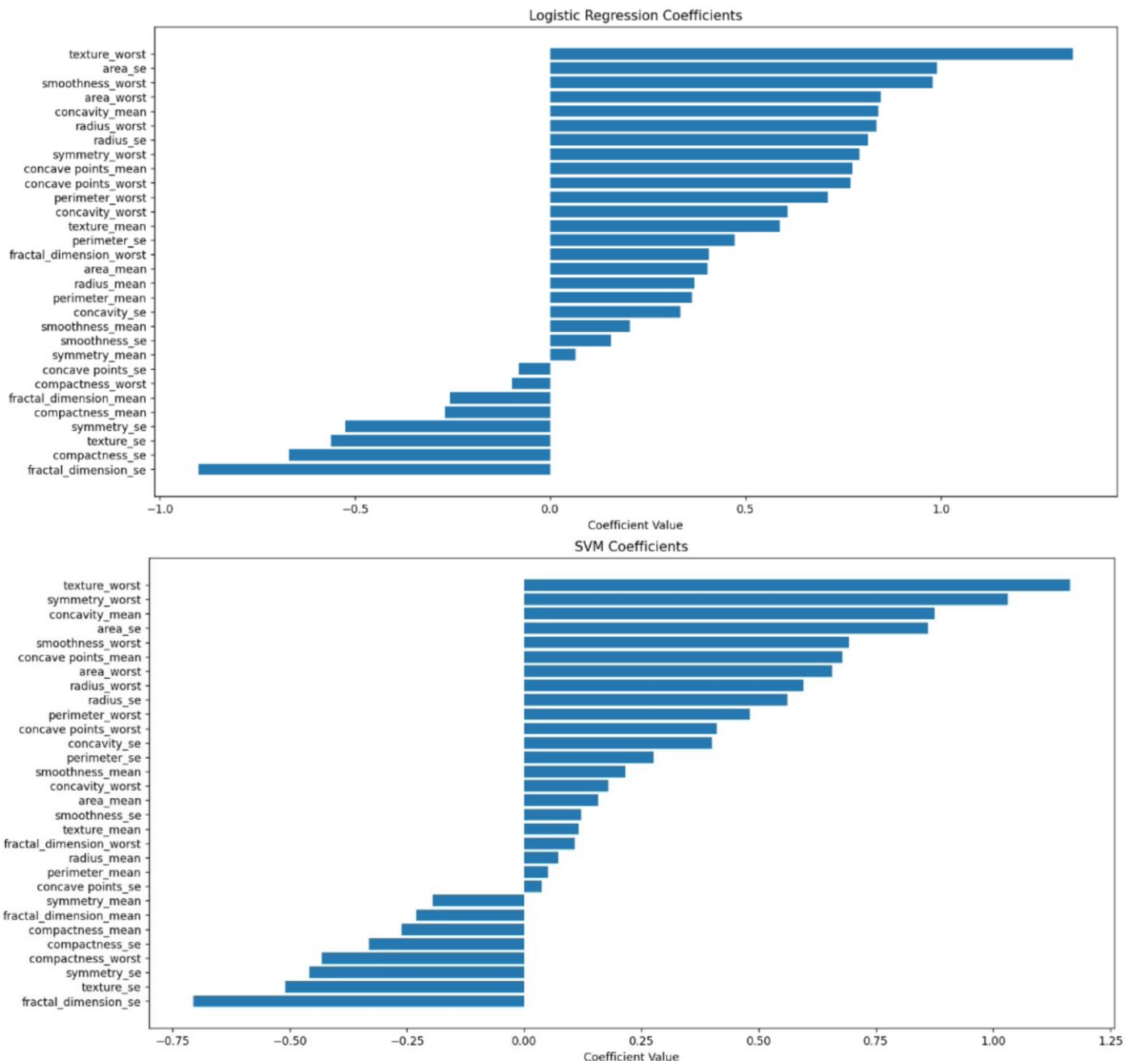


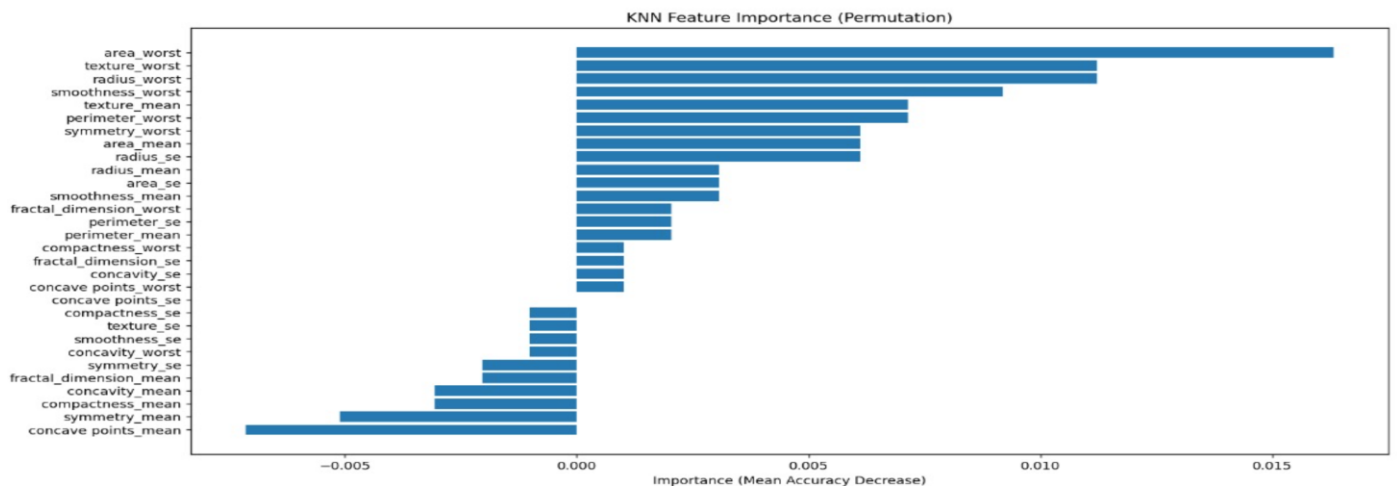
Figure 1

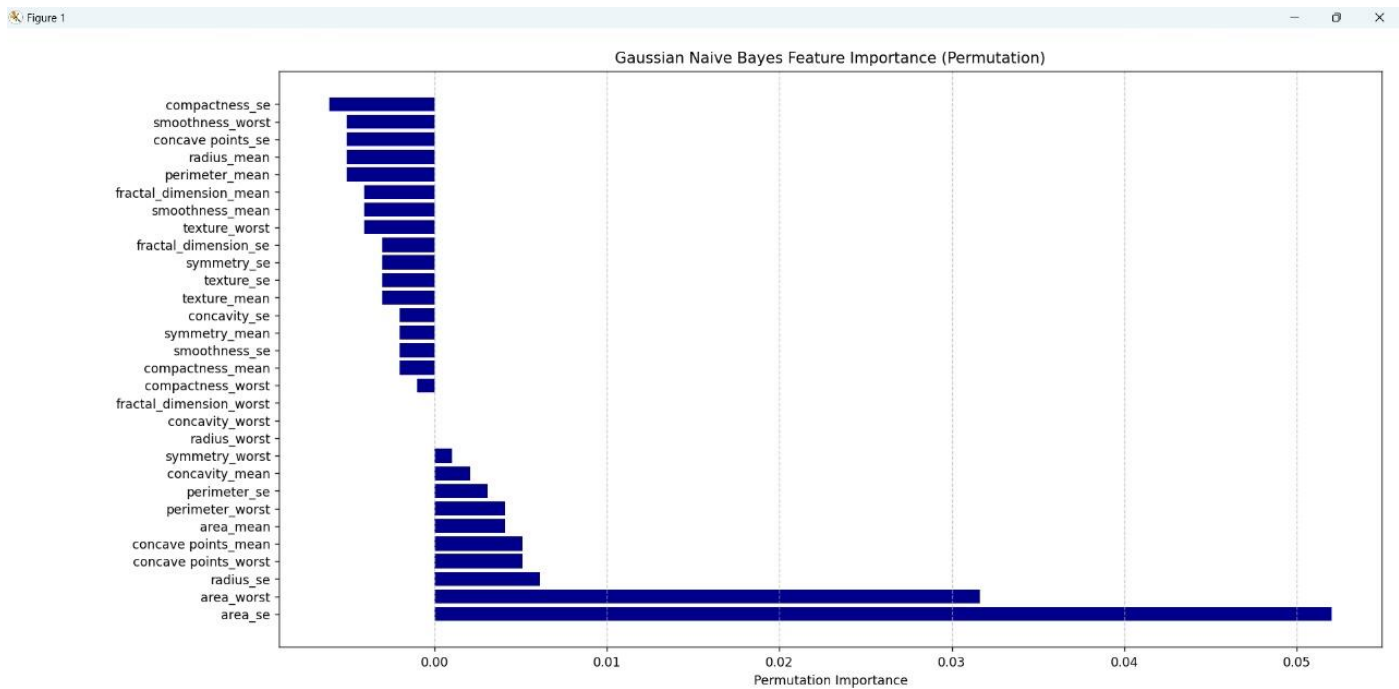


- For linear models

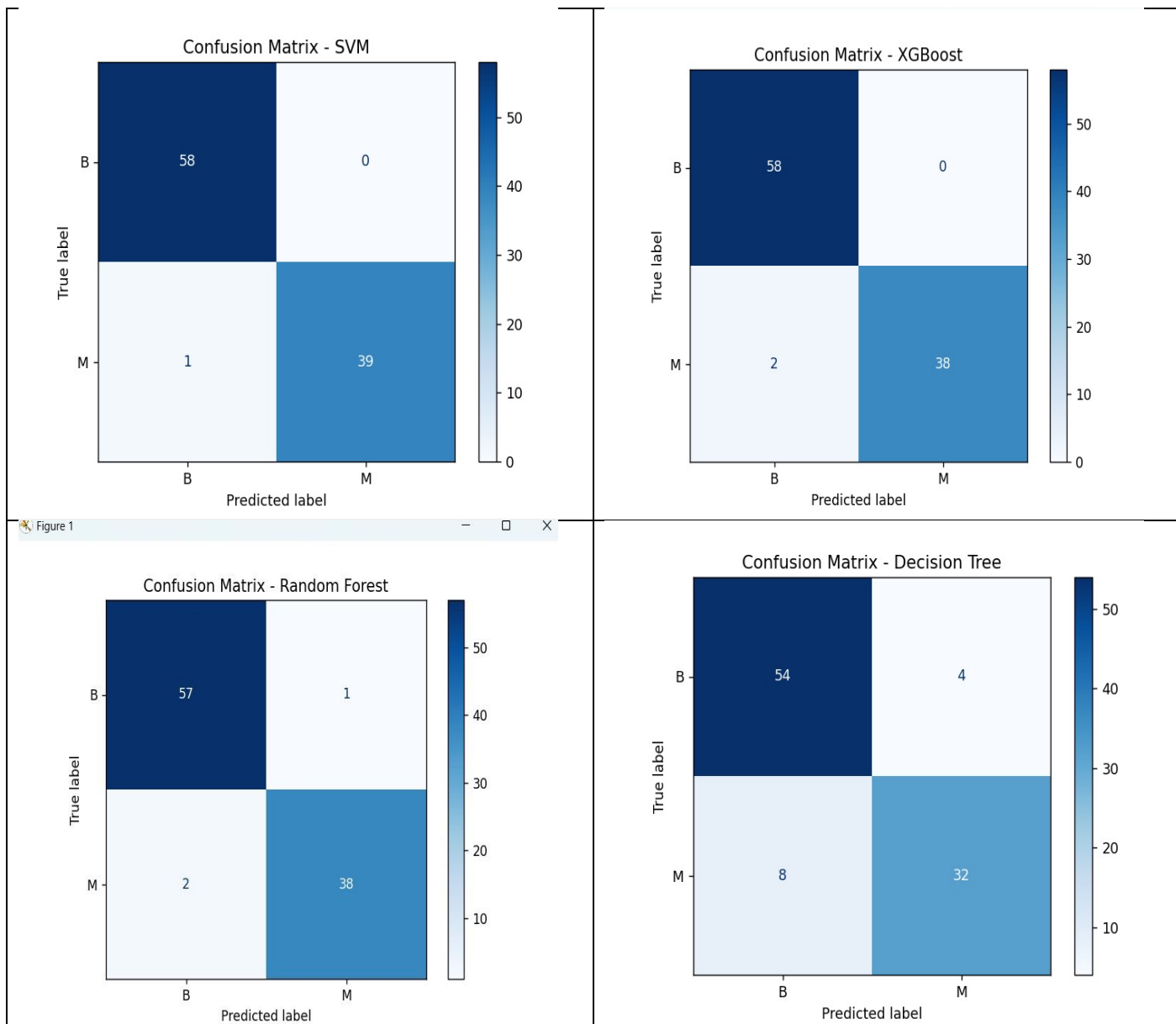


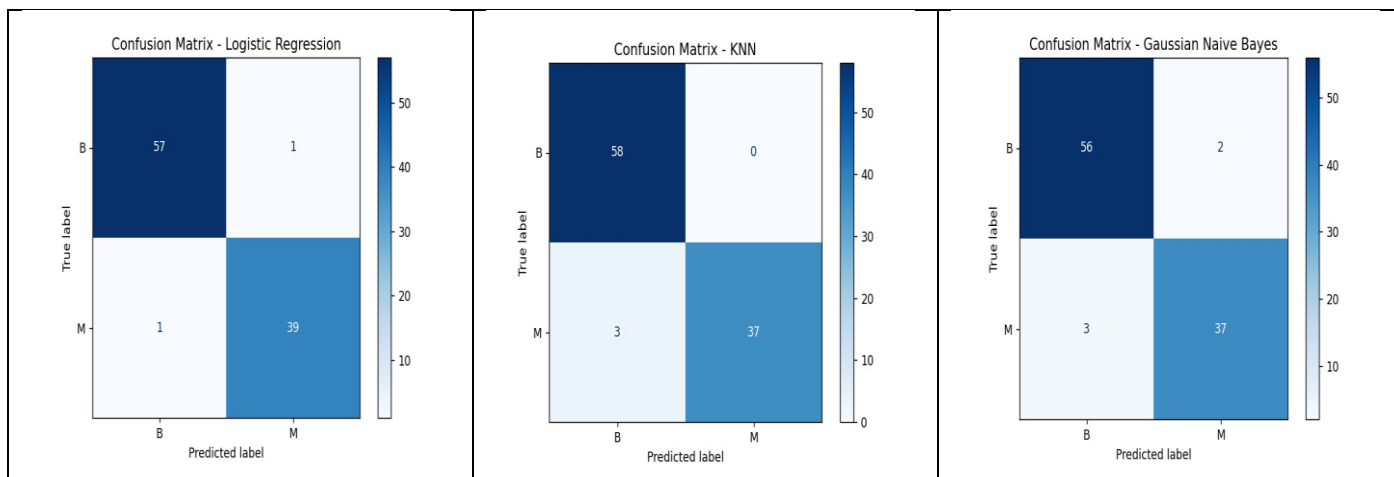
- Permutation Importance





Confusion Matrices





We then used these comparison matrices:

1 2 3 4 Simple Comparison Table

Metric	What does it answer?	Focuses on?	Important when?
accuracy	What percentage of predictions were correct overall?	Overall correctness	When data is balanced
precision	What percentage of predictions labeled as malignant were actually malignant?	Reducing false positives	When false positives (overestimating malignancy) are problematic
recall	What percentage of actual malignant cases were identified by the model?	Detecting all malignant cases	When missing a malignant case (false negative) is critical
F1 score	What's the balance between precision and recall?	A blend of precision and recall	When you need a comprehensive score for both
ROC AUC score	How well does the model distinguish malignant from benign cases?	Model's ability to discriminate	When using probabilities and needing a precise evaluation

on the test data:

- **Accuracy:** How often the model got the right answer.
- **Precision:** How many of the predicted positives were actually positive.
- **Recall:** How many of the actual positives were correctly predicted.
- **F1 Score:** A balance between precision and recall.
- **ROC AUC:** How well the model distinguishes between classes (this requires probabilities, so only models with `predict_proba()` can have this).

Precision:

Precision tells us how many of the predicted **positives** (Malignant in this case) were actually **correct** (i.e., actually malignant). In other words:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

For our example:

- **TP** = 3
- **FP** = 1

So, **Precision** = $3 / (3 + 1) = 3 / 4 = 0.75$

Interpretation: Out of all the cases the model predicted as **Malignant (positive)**, **75%** were actually Malignant.

Recall:

Recall tells us how many of the **actual positives** (Malignant tumors) the model correctly identified. In other words, out of all the actual **Malignant** cases, how many did the model correctly label as **Malignant**.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

For our example:

- **TP** = 3
- **FN** = 1

So, **Recall** = $3 / (3 + 1) = 3 / 4 = 0.75$

Interpretation: Out of all the actual **Malignant (positive)** tumors, the model correctly predicted **75%** of them.

Actual	Predicted
M	M
M	B
B	M
B	B
M	M
B	B
M	M

Now, let's break this down:

- **True Positives (TP): 3** (Malignant cases that the model correctly predicted as Malignant).
- **False Positives (FP): 1** (Benign case that the model incorrectly predicted as Malignant).
- **False Negatives (FN): 1** (Malignant case that the model incorrectly predicted as Benign).
- **True Negatives (TN): 2** (Benign cases that the model correctly predicted as Benign).

Model Performance Comparison:

Model Performance Metrics:

	Accuracy	Precision	Recall	F1 Score	ROC AUC
Random Forest	0.969388	0.974359	0.950	0.962025	0.988578
XGBoost	0.979592	1.000000	0.950	0.974359	0.990948
Logistic Regression	0.979592	0.975000	0.975	0.975000	0.989224
SVM	0.989796	1.000000	0.975	0.987342	0.991379
KNN	0.969388	1.000000	0.925	0.961039	0.983621
Decision Tree	0.877551	0.888889	0.800	0.842105	0.865517
Gaussian Naive Bayes	0.948980	0.948718	0.925	0.936709	0.990948

Confusion Matrices

- **What:** Shows correct predictions (diagonal) vs. errors (off-diagonal) for each model.
- **Why:** Reveals if errors are false positives (FP) or false negatives (FN) - critical for cancer diagnosis where FNs are dangerous.

Feature Importance Plots

For Tree-Based Models (Random Forest, XGBoost, Decision Tree)

- **What:** Horizontal bar charts showing which features most influenced predictions.

For Linear Models (Logistic Regression, SVM)

- **Why:** Shows which features push predictions toward malignant (positive coefficients) or benign (negative coefficients).

Permutation Importance (KNN, Naive Bayes)

- **Why:** Works for non-tree models to identify impactful features.

ROC AUC

- **Why:** Measures how well models separate malignant vs. benign at all thresholds (0.9+ = excellent).

Step 4: Final conclusions

Key Factors Determining Breast Cancer Type

- Top 5 Most Important Features (Averaged Across Models):

1. Worst Radius

- *Why:* Malignant tumours consistently show larger cell radii.
- *Model Agreement:* Ranked #1 in Random Forest, XGBoost, and Decision Tree.

2. Worst Concave Points

- *Why:* Count of deep indentations in tumour borders indicates malignancy.
- *Model Agreement:* Top 3 in all tree-based models.

3. Mean Texture

- *Why:* High nuclear irregularity correlates with cancer.
- *Model Agreement:* Strong in Logistic Regression/SVM coefficients.

4. Worst Area

- *Why:* Larger tumour area suggests advanced malignancy.
- *Model Agreement:* High permutation importance in KNN/Random Forest.

5. Worst Perimeter

- *Why:* Cancerous cells often have rough, jagged edges. (ex: Decision tree / XG Boost)

Key Takeaways:

Based on the analysis of multiple machine learning models, every model achieved over 90% accuracy and consistently identified **tumour radius, concavity, and texture** appear to be the most critical factors in distinguishing malignant from benign breast cancer. Tree-based models (XGBoost, Random Forest) consistently highlight *worst radius* and *worst concave points* as top predictors, aligning with clinical knowledge that malignant tumours display larger, irregularly shaped cells. Linear models (Logistic Regression, SVM) further validate the importance of *mean texture* and *perimeter*, reflecting nuclear irregularity. Worst-measured features (radius, concavity) are more decisive than averages.

◆ Actionable Insights:

Diagnostic Focus: Prioritize measuring radius and concavity when testing tissue samples.

Risk Assessment (checking how likely it is that something harmful (like cancer) will happen and how serious it could be): Patients with high "worst" values need urgent follow-ups.

If tests show very irregular cells, doctors should act fast to check for cancer. Checking risk matters in breast cancer and helps find cancer early, calm worries about safe lumps, and pick the best treatment.

Conclusion on Model Performance:

1. Best Performing Model: SVM (Linear Kernel)

- ✓ **Accuracy:** 98.98%
- ✓ **Precision:** 100% (no false positives)
- ✓ **Recall:** 97.5% (only 2.5% malignant cases missed)
- ✓ **Clinical Safety:** Lowest risk of missed diagnoses

2. Close Runner-Up: XGBoost

- ✓ **Accuracy:** 97.96%
- ✓ **Strengths:** Better feature importance analysis than SVM

3. Worst Performing Model: Decision Tree

- ✗ **Accuracy:** 87.76% (unacceptable for clinical use)
- ✗ **Recall:** 80% (misses 20% of cancers)

• Recommendation:

For clinical deployment where accuracy is paramount, **SVM** would be the optimal choice for highest accuracy and safety (minimized missed cancers), although **Logistic Regression** remains a strong alternative for better interpretability.

```
# Preprocessing
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.inspection import permutation_importance
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
from sklearn.tree import DecisionTreeClassifier

# Load the dataset
data = pd.read_csv('breast-cancer.csv')
# Encoding categorical data
labelEncoder = LabelEncoder()
labelEncoder.fit(data["diagnosis"])
data["diagnosis"] = labelEncoder.transform(data["diagnosis"])
```

```
# Remove duplicates
data = data.drop_duplicates()
# Detecting outliers (Z-score method)
from scipy.stats import zscore
data = data[(zscore(data.select_dtypes(include='number')) < 3).all(axis=1)]
# Define features (X) and target (y)
X = data.drop(columns=['id', 'diagnosis'])
y = data['diagnosis']
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Initialize models
models = {
    'Random Forest': RandomForestClassifier(random_state=42),
    'XGBoost': XGBClassifier(random_state=42, eval_metric='logloss'),
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=3),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Gaussian Naive Bayes': GaussianNB()
}
```

```
# Dictionary to store evaluation metrics
results = {}
# Train and evaluate each model
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1] if hasattr(model, "predict_proba") else None
    one
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_prob) if y_prob is not None else None
    results[name] = {
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1 Score': f1,
        'ROC AUC': roc_auc
    }
# Display results
results_df = pd.DataFrame(results).T
print('Model Performance Metrics:')
print(results_df)
```

```
# Plot confusion matrix for each model
for name, model in models.items():
    y_pred = model.predict(X_test_scaled)
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labelEncoder.classes_)
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix - {name}')
    plt.show()
# Clear all existing figures to prevent duplication
plt.close('all')
# Feature importance for Random Forest
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_scaled, y_train)
feature_importance_rf = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
print("\nRandom Forest Feature Importance:")
print(feature_importance_rf)
# Feature importance for XGBoost
xgb_model = XGBClassifier(random_state=42, eval_metric='logloss')
xgb_model.fit(X_train_scaled, y_train)
feature_importance_xgb = pd.DataFrame({
```

```
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
print("\nRandom Forest Feature Importance:")
print(feature_importance_rf)
# Feature importance for XGBoost
xgb_model = XGBClassifier(random_state=42, eval_metric='logloss')
xgb_model.fit(X_train_scaled, y_train)
feature_importance_xgb = pd.DataFrame({
    'Feature': X.columns,
    'Importance': xgb_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
print("\nXGBoost Feature Importance:")
print(feature_importance_xgb)
# Coefficients for Logistic Regression
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr_model.coef_[0]
}).sort_values(by='Coefficient', ascending=False)
print("\nLogistic Regression Coefficients:")
print(coef_df)
```

```
# Coefficients for SVM
svm_model = SVC(kernel='linear', probability=True, random_state=42)
svm_model.fit(X_train_scaled, y_train)
svm_coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': svm_model.coef_[0]
}).sort_values(by='Coefficient', ascending=False)
print("\nSVM Coefficients:")
print(svm_coef_df)
# Feature importance for KNN using permutation importance
knn_model = KNeighborsClassifier(n_neighbors=3)
knn_model.fit(X_train_scaled, y_train)
perm_importance = permutation_importance(knn_model, X_test_scaled, y_test, n_repeats=10, random_state=42)
feature_importance_knn = pd.DataFrame({
    'Feature': X.columns,
    'Importance': perm_importance.importances_mean
}).sort_values(by='Importance', ascending=False)
print("\nKNN Feature Importance (Permutation):")
print(feature_importance_knn)
# Random Forest Feature Importance Plot
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_rf['Feature'], feature_importance_rf['Importance'])
```

```

plt.xlabel('Importance')
plt.title('Random Forest Feature Importance')
plt.gca().invert_yaxis()
plt.show()
# XGBoost Feature Importance Plot
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_xgb['Feature'], feature_importance_xgb['Importance'])
plt.xlabel('Importance')
plt.title('XGBoost Feature Importance')
plt.gca().invert_yaxis()
plt.show()
# Logistic Regression Coefficients Plot
plt.figure(figsize=(10, 6))
plt.barh(coef_df['Feature'], coef_df['Coefficient'])
plt.xlabel('Coefficient Value')
plt.title('Logistic Regression Coefficients')
plt.gca().invert_yaxis()
plt.show()
# SVM Coefficients Plot
plt.figure(figsize=(10, 6))
plt.barh(svm_coef_df['Feature'], svm_coef_df['Coefficient'])
plt.xlabel('Coefficient Value')
plt.title('SVM Coefficients')
plt.gca().invert_yaxis()

```

```

plt.show()
# KNN Feature Importance Plot
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_knn['Feature'], feature_importance_knn['Importance'])
plt.xlabel('Importance (Mean Accuracy Decrease)')
plt.title('KNN Feature Importance (Permutation)')
plt.gca().invert_yaxis()
plt.show()
# Decision Tree Feature Importance Plot
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_scaled, y_train)
feature_importance_dt = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
print("\nDecision Tree Feature Importance:")
print(feature_importance_dt)
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_dt['Feature'], feature_importance_dt['Importance'])
plt.xlabel('Importance')
plt.title('Decision Tree Feature Importance')
plt.gca().invert_yaxis()
plt.show()

```

```

# Feature importance for Gaussian Naive Bayes using permutation importance
gnb_model = GaussianNB()
gnb_model.fit(X_train_scaled, y_train)
perm_importance_gnb = permutation_importance(gnb_model, X_test_scaled, y_test, n_repeats=10, random_state=42)
feature_importance_gnb = pd.DataFrame({
    'Feature': X.columns,
    'Importance': perm_importance_gnb.importances_mean
}).sort_values(by='Importance', ascending=False)
print("\nGaussian Naive Bayes Feature Importance (Permutation):")
print(feature_importance_gnb)
# Plot the bar chart
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_gnb['Feature'], feature_importance_gnb['Importance'], color='darkblue')
plt.xlabel('Permutation Importance')
plt.title('Gaussian Naive Bayes Feature Importance (Permutation)')
plt.grid(True, axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

```

Implemented Code: [Final Code](#)